

Makeshift Automated Patching for Binary Programs

Nguyễn Gia Phong 한승헌 이주용

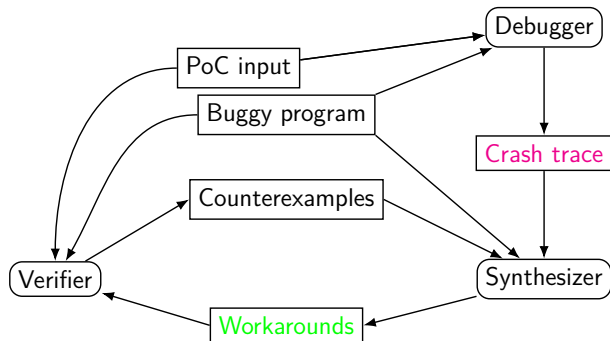
Lab of Software, UNIST

2026-07-14

- ▶ Bugs reproduction is complex, relying on build configurations, environments inputs, etc.
- ▶ Software in production may lack source code or reproducible build
- ▶ Common machine codes are unstructured
→ Imperfect binary patches as workarounds

Patch format

- ▶ Patch is inserted as a C-like ABI trampoline
`@patch location`
`+if predicate`
`+     bailout`
- ▶ `patch location` is on the crash's stack trace
- ▶ `predicate`'s operands are registers and memory
- ▶ `bailout` is a jump to `return/exit`



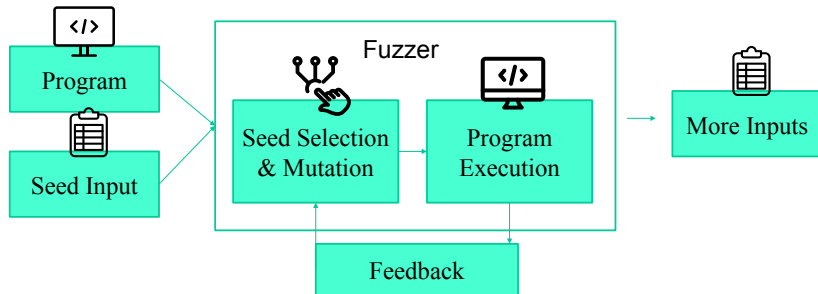
- ▶ Operand value capture
 - ▶ At patch location
 - ▶ Snapshot along call stack
- ▶ Template-based
- ▶ Heuristics for frequent CWEs, e.g. 0^{-1} , *NULL
- ▶ Fallbacks: ERM, LLM

Verification...

Input \ Program	Original	Patched
	Original	Patched
Malicious		
Benign		

 $\neq$  $\rightarrow$ overfit patch

... with system-level inputs

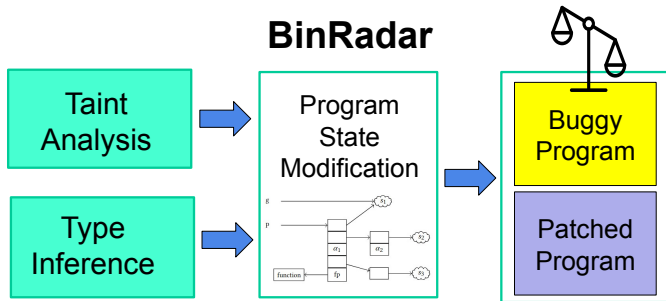


- ▶ Derived from PoC via fuzzing or concolic execution
- ▶ May fail to distinguish correct and plausible patches, e.g. when all generated inputs are malicious

... with function-level inputs

Under-constrained concolic execution, at the patch's function entry:

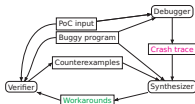
- ▶ Fork and modify program states
- ▶ Limitation: infeasible states



Motivation

- Bugs reproduction is complex, relying on build configurations, environments inputs, etc.
- Software in production may lack source code or reproducible build
- Common machine codes are unstructured
→ Imperfect binary patches as workarounds

Architecture



Patch format

- Patch is inserted as a C-like ABI trampoline
@patch location
+if predicate
+ bailout
- patch location is on the crash's stack trace
- predicate is an expression of registers and memory values
- bailout is a jump to return/exit block

Patch & bailout localization

- Predicate is commonly of next insn's operands
- Production binary
 - Sanitizers in emulator, less effective
 - Incomplete disassembly for CFG

Value capture

- Naive approach: values alive at patch location
- Optimizing compilers prefer register allocation
- Function signature cannot be patched
→ Capture snapshot along call stack

Predicate synthesis strategies

- Heuristics for frequent CWEs:
div by 0, NULL deref, buffer overflow
- Template with empirical risk minimization
- Fall back to LLM-generated C trampoline

Goals

- ✓ Zero-day vulnerability mitigation
- ✓ Applicable to stripped binaries
- ✗ Human-readable patches
- ✗ Semantic perfection, e.g.
 - Complete mitigation/lack of regression
 - Proper resource clean-up/error reporting

Verification

	Program	
Input	Original	Patched
Malicious	⊗	⊗
Benign	⊗	⊗

⊗ ≠ ⊗ → overfit patch

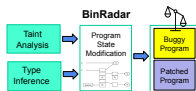
System-level inputs

- Concrete inputs derived from PoC via fuzzing or concolic execution
- Limitation: may fail to distinguish correct and plausible patches, e.g. when all generated inputs are malicious

Function-level inputs

- Modified program states at the entry of the patch's function, i.e. in under-constrained concolic execution
- Limitation: infeasible states

Program state modification



- Preserve the clean state by forking the process at the function's entry
- Modify it according to
 - Taint analysis from PoC input
 - Type inference
- Run combined original and patched programs
- Compare their behavior

Questions?
Comments?
Come visit us!